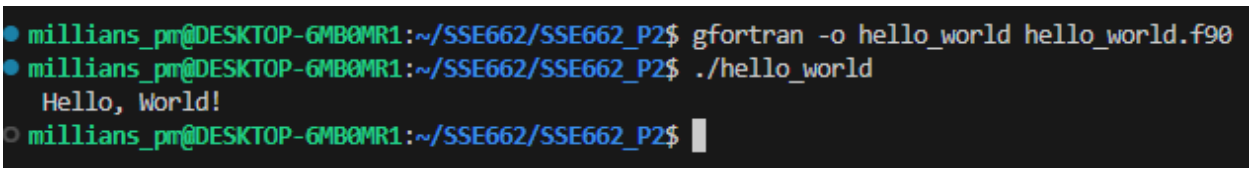
The Complex Number Calculator project, implemented in FORTRAN, required the development of a FORTRAN skillset. While the initial “hello world” module proved incredibly easy, that part of the assignment did not foreshadow the ease of the rest of the project.

The setup of the FORTRAN environment consisted of installing a linux FORTRAN binary to use on my Ubuntu virtual machine. I found that this binary was the most straightforward, especially because I had already set up GitHub and VSCode on the VM. After creating a GitHub repo for the project, I simply pulled the branch to the VM and began to set up the project. Unlike Go in the last project, FORTRAN allows typical directory structures, with all source files in the same folder. Thus, I was able to create a “docs” folder with design files, a “src” folder with my source files, and a “tests” folder with test scripts. In my top level, I included a “README” markdown file to describe the project usage, along with a makefile to compile the project with ease. Using this layout, design, implementation, testing, and compilation were straightforward.

The “hello world” part of the project essentially just tested the FORTRAN compiler to ensure that the environment was set up correctly. The following image depicts the compilation and running of the "hello world” module.



The Complex Number Calculator assignment, however, came with unanticipated difficulties. The primary challenge in the design of the calculator was the command-line interface construction. Due to the requirement of multiple inputs by the user – command, complex number, complex number – the decision of whether to require three separate inputs or a single line had to be made. The final decision was to incorporate a single line command consisting of a command followed by the operands of the command. After this decision was made, no other design challenges presented themselves; however, mysterious bugs began to arise. Unlike any language I am familiar with, strings in FORTRAN have set lengths that they cannot exceed. This, at first, did not bother me because I could just set the length to a high, arbitrary value which the string would never exceed (I chose 256 for most strings). However, this approach would fail when concatenating strings. When printing a complex number, I would print the real number and imaginary numbers separated by the sign of the imaginary number, for instance “4 + 3i”. However, I had stored the “ + “ as a 256-length string. Therefore, the print string would be cut off after the “ + “

because it was too big. This issue was much easier to spot than the next issue, which costed several hours of debugging.

During automated testing, I created tests with various use cases for each utility method. My utility methods included a series of methods and submethods to parse a string to find the real and imaginary numbers inside the string. For instance, the input “4+2i” would output floats with real = 4 and imag = 2. All of my test cases worked, even when testing each input format and positive and negative values. However, when testing the Calculate method which utilized these utility functions, I began to see unexplainable behavior. When inputting the same complex numbers that worked in the utility method testing, the Calculate test output would include parts of the correct answer preceding random characters and symbols. When debugging, I found that this issue was happening **inside** the utility methods, which made no sense to me as I had just tested these methods thoroughly. For several hours, I tried to debug this issue with print statements and alternative cases before finally realizing that the string length issue had infiltrated the Calculate function. Each input complex number string was set to only 20 characters, so these 20 characters would pass into the utility method, and the random bytes stored at the next 236 memory addresses would leak into the function. I have never seen a problem like this in any language I have used, so it was a very frustrating finding.

Potential improvements for the calculator could include more robust string processing. For instance, I am someone who prefers to add spaces between operators; I would prefer to input a complex number in the form “4 + 2i” or “(-1, 0).” The current input string processor splits the pieces by space characters, so this would not work currently. However, this improvement would allow more flexibility in the input. Additionally, though the project specifies a command-line interface, I believe any calculator should be implemented through a GUI. Besides those potential improvements, I think the complex number calculator has a developed and strong implementation.